Guía del K++ Bootstrap Studio

El mini-IDE para compilar, probar y ejecutar programas K++ en KolibriOS.

Proyecto K++ · Esequiel Pena Perez

Segunda edición · Noviembre de 2026

Esta edición incorpora el ciclo completo de KolibriOS (MOVER BINARIO y EJECUTAR), las guardas contra errores humanos y el registro de causas de error, todo ello añadido al Studio el 2026-11-11. La primera edición es de octubre de 2026.

1. ¿Qué es el Bootstrap Studio?

El **K++ Bootstrap Studio** (`kpp_bootstrap_studio.py`) es un mini-IDE gráfico escrito en Python con Tkinter, pensado para trabajar con el compilador K++ sin salir de una sola ventana. Su flujo de trabajo cubre el ciclo entero de un programa K++, del fuente a KolibriOS:

```
EDITOR → COMPILAR → RAW / BIN → DESENSAMBLADO (objdump i386)
                              ↓
                MOVER BINARIO → imagen de KolibriOS → EJECUTAR (QEMU)
```

Con él puedes escribir o abrir un programa `.kpp`, compilarlo, ver la salida del compilador (incluido el AST), inspeccionar el código máquina x86 generado, **instalar el ejecutable dentro de una imagen de KolibriOS** — con su nombre 8.3 y su registro en `AUTORUN.DAT` — y **arrancar KolibriOS en QEMU** para verlo correr. Es la herramienta recomendada para testear los ejemplos del proyecto y para las pruebas de campo de Stage 1.

El Studio no es un programa aparte del compilador: por debajo invoca a `bin/kpp`, a `objdump` y a las herramientas `mtools`, y conoce las reglas del banco de KolibriOS (la imagen persistente, el desplazamiento de la partición FAT32, el formato de `AUTORUN.DAT`) porque las heredó del Mini Entorno ASM del mismo autor.

2. Requisitos

Requisito	Para qué	Cómo verificarlo
Linux (Debian / MX Linux, o WSL)	Entorno del proyecto	uname -a
Python 3	Ejecuta el Studio	python3 --version
Tkinter (python3-tk)	La interfaz gráfica	python3 -c "import tkinter"
Compilador C++17 + CMake ≥ 3.16	Construir bin/kpp	g++ --version; cmake --version
objdump (binutils)	La pestaña de desensamblado	objdump --version
dd, cksum (coreutils)	Separar la cabecera; calcular el nombre KolibriOS	dd --version
mtools (mcopy, mdir, mtype)	Mover binarios dentro de la imagen	mdir --version
qemu-system-x86 (qemu-system-i386)	El botón EJECUTAR	qemu-system-i386 --version
Una imagen persistente de KolibriOS	El disco donde viven los programas	Ver §3

En Debian/MX Linux, si falta algo: `sudo apt install python3-tk build-essential cmake binutils mtools qemu-system-x86`.

3. Instalación y configuración

Paso 1 — Clona el repositorio y compila el compilador K++

```
git clone https://github.com/saykol31/KPP.git
cd KPP
cmake -S . -B build
cmake --build build
```

Esto genera el ejecutable `bin/kpp`, que es lo que el Studio invoca por debajo.

Paso 2 — Nada que ajustar: el Studio se encuentra solo

El Studio localiza la raíz del repositorio a partir de su propia ubicación, y de ahí deriva todo lo demás: `bin/kpp`, la carpeta `projects/` y la carpeta de salida `Output/`. Si alguna vez necesitas forzarlo, la variable de entorno `KPP_ROOT` manda sobre la detección automática. (En la primera edición había que editar la ruta a mano dentro del archivo; ya no.)

Paso 3 — La imagen de KolibriOS

MOVER BINARIO y EJECUTAR trabajan sobre una **imagen persistente** de KolibriOS: una imagen cruda de disco cuya partición FAT32 empieza en el sector 2048, como las oficiales. Por defecto el Studio busca:

```
$HOME/VirtualBox VMs/kolibriOS/KPP-KOLIBRI.img
```

Si tu imagen vive en otra ruta, díselo con la variable de entorno `KPP_KOLIBRIOS_IMAGE` (por ejemplo, en tu `~/.bashrc`):

```
export KPP_KOLIBRIOS_IMAGE="$HOME/Home/VirtualBox VMs/kolibriOS/KPP-KOLIBRI.img"
```

¿No tienes imagen? Descarga la construcción oficial en formato **raw** desde `builds.kolibrios.org`, descomprime el `.7z` y renombra `kolibri.raw` a `KPP-KOLIBRI.img` en la ruta de arriba. La imagen es un banco de pruebas regenerable: tus fuentes viven en el repo, no en ella.

Paso 4 — Lanza el Studio

```
python3 kpp_bootstrap_studio.py
```

Al abrir, el Studio carga por defecto el lexer de Stage 1 (`compiler/kpp/lexer/Lexer.kpp`), porque es la pieza en curso del proyecto.

El error más común: *si al compilar aparece `[ERROR] No existe K++: .../bin/kpp`, el Paso 1 no se completó. Si al mover aparece que no existe la imagen, revisa el Paso 3.*

4. Tour de la interfaz

La ventana se divide en tres zonas:

Barra superior de herramientas

Botón	Qué hace
COMPILAR	Compila el archivo actual con bin/kpp y muestra la salida.
RUN	Pipeline completo: compila → genera RAW/BIN → extrae el payload → lo desensambla con objdump.
MOVER BINARIO	Instala el .bin del programa actual dentro de la imagen de KolibriOS: nombre 8.3, copia y registro en AUTORUN.DAT (§9).
EJECUTAR	Arranca KolibriOS en QEMU con la imagen persistente (§9). Antes comprueba sus guardas (§10).
DEBUG	Reservado: el depurador llegará después del bootstrap (§12).
GUARDAR	Guarda el contenido del editor en el archivo actual.
ABRIR	Abre un archivo .kpp.

Panel izquierdo: editor K++ SOURCE

Editor con números de línea y resaltado básico. La barra inferior muestra `Ln / Col` (línea y columna del cursor) y el indicador de estado (`LISTO`, `COMPILANDO...`, etc.).

Panel derecho: pestañas de resultados

Pestaña	Contenido
K++ OUTPUT	Salida estándar del compilador: cabecera K++ COMPILER, el AST del programa, errores de léxico/sintaxis/semántica y confirmaciones de los escritores de binario.
DISASSEMBLY	Desensamblado del código máquina generado (objdump, CPU i386, sintaxis Intel). Conmutadores CODE / DATA / RAW para cambiar la vista.

Barra inferior: SISTEMA / NOTIFICACIONES

El registro de eventos de la sesión, y la pieza que más trabaja de la ventana: ahí cae el veredicto de cada compilación (con el nombre KolibriOS del programa, §5), los seis pasos de cada movimiento a la imagen (§9), las guardas cuando se niegan a actuar (§10) y, junto a cada error del ciclo KolibriOS, la línea `[DETALLE]` con las palabras de la propia herramienta que falló.

Los estados que verás con más frecuencia:

Estado	Significado
RUN OK	El pipeline completo terminó bien.
BOOTSTRAP OK	Se generaron RAW y BIN correctamente.
KOLIBRIOS LISTO	El binario quedó instalado en la imagen y registrado.
ERROR	Algo falló; el detalle está en el propio registro o en K++ OUTPUT.
DISASM ERROR	El binario se generó pero el desensamblado falló.
BIN INVÁLIDO	El archivo generado no tiene el formato esperado.
SALIDA INCOMPLETA	Faltó alguno de los artefactos esperados.

5. Tu primera compilación

1. Abre el Studio y pulsa **ABRIR**: elige `projects/L01.02_hola.kpp`, un programa mínimo de la batería.

2. Pulsa **RUN**.

3. Observa la pestaña **K++ OUTPUT**: verás la cabecera del compilador, el AST (`Program → Function: main`) y la confirmación del ejecutable.

4. Cambia a la pestaña **DISASSEMBLY**: ahí está tu programa traducido a ensamblador x86 de 32 bits.

5. En **SISTEMA / NOTIFICACIONES** verás el cierre de la compilación:

```
============================================================
BOOTSTRAP COMPLETADO.
[SOURCE] L01.02_hola.kpp
[BIN]    /home/tu_usuario/KPP/Output/L01.02_hola.bin
[KOLIBRIOS] L01_02DD (así se verá dentro de KolibriOS)
============================================================
```

La línea `[KOLIBRIOS]` es importante: el archivo **cambia de nombre** antes de entrar en KolibriOS (§8), y esa línea te dice en cada compilación cómo se llamará el tuyo allá dentro. Cuando la imagen se llena de programas, esa pareja fuente → nombre es la que te mantiene orientado.

La diferencia entre los dos botones de compilar:

- **COMPILAR** solo ejecuta `./bin/kpp archivo.kpp` y te muestra la salida. Ideal para iterar rápido mientras escribes.
- **RUN** hace todo el pipeline: compila, genera `.raw` y `.bin`, recorta la cabecera de 36 bytes con `dd` y desensambla el payload con `objdump -m i386` en sintaxis Intel. Es lo que quieres para testear ejemplos.

6. Cómo testear los ejemplos del proyecto

El repositorio trae dos colecciones de programas:

- `projects/` — la batería del lenguaje, la **serie L** (`L00`–`L18`, con inserciones decimales y sufijo `_neg` para las pruebas que el compilador debe rechazar). Es la que valida la suite automática.
- `compiler/kpp/` — **Stage 1**, el compilador reescrito en K++. Sus programas de prueba forman la **serie P**; el primero, P00, es el archivo `FUENTE.KPP` que el lexer lee al correr en KolibriOS.

La batería entera se valida desde la terminal, sin abrir el Studio:

```
./validar_test.sh
```

Compila cada `.kpp`, comprueba que los negativos sean rechazados y resume: en el estado actual del proyecto, **60 positivos, 11 negativos, 0 fallos**. El Studio es para el trabajo de a uno: abrir un ejemplo, compilarlo, auditar su desensamblado y llevarlo a KolibriOS en el ciclo de §9.

Ruta sugerida dentro del Studio, de fácil a avanzado:

#	Ejercicio	Qué compruebas
1	L01.02_hola.kpp	El ciclo mínimo: compilar, mover, ejecutar
2	L02_variables.kpp	Variables y su salida en KolibriOS
3	L03.01_expresiones.kpp	Funciones y expresiones; el resultado se imprime una vez, donde el programa hace print_int
4	L05_parametros.kpp	Parámetros y llamadas
5	L15_recursion.kpp	Recursión con marco de pila
6	L18_file_io_bloques.kpp	Entrada/salida de archivos por bloques
7	compiler/kpp/lexer/Lexer.kpp	Stage 1 en campo: su sujeto es P00 (FUENTE.KPP)

***Las pruebas negativas** (sufijo `_neg`) están hechas para que el compilador las rechace: en el Studio verás el error en K++ OUTPUT, que es exactamente lo que deben producir. La lista oficial de negativos vive dentro de `validar_test.sh` y en `compiler/kpp/TESTS.md` junto al registro de la serie P.*

7. Leyendo el desensamblado

La pestaña **DISASSEMBLY** muestra la salida de `objdump` sobre el payload del binario (el archivo `.bin` menos sus 36 bytes de cabecera). La cabecera de la pestaña indica el archivo, `CPU: i386` y `Syntax: Intel`.

Los tres conmutadores de vista:

Vista	Muestra	Úsala para
CODE	El código máquina como instrucciones ensamblador	Verificar que tus funciones generan el x86 esperado (call, ret, accesos a pila…)
DATA	La zona de datos del binario	Localizar literales de texto y constantes
RAW	Volcado crudo del payload	Inspección byte a byte cuando algo no cuadra

Para qué sirve en la práctica: *si un ejemplo compila pero sospechas que el binario hace algo raro, el desensamblado te deja auditar lo que realmente se generó, instrucción por instrucción, antes de llevarlo a KolibriOS.*

8. Archivos generados y el nombre KolibriOS

Cada compilación exitosa deja dos artefactos en la carpeta `Output/`, nombrados según tu archivo fuente (por ejemplo, `hola.kpp` → `hola.raw` y `hola.bin`):

Archivo	Qué es
.raw	Artefacto intermedio del pipeline K++.
.bin	Ejecutable nativo de KolibriOS (x86 de 32 bits, con cabecera MENUET01 de 36 bytes).

Dentro de KolibriOS el ejecutable **no lleva extensión** y se reconoce por su cabecera, no por el nombre. Pero el nombre sí importa, porque la imagen es FAT y rige el formato 8.3. El Studio calcula el nombre KolibriOS de cada programa con la misma regla del Mini Entorno ASM:

1. Se toma el nombre del fuente sin extensión, en mayúsculas.
2. Todo carácter fuera de `A-Z 0-9 _ $ -` se convierte en `_`.
3. Si el resultado pasa de 8 caracteres, se queda con los 6 primeros más un sufijo de dos dígitos hexadecimales: la suma de verificación (`cksum`) del nombre original, módulo 256.

Fuente	Nombre en KolibriOS
Lexer.kpp	LEXER
L01.02_hola.kpp	L01_02DD
L02_variables.kpp	L02_VA49
L03.01_expresiones.kpp	L03_016C

El sufijo hace únicos a los nombres largos que comparten los 6 primeros caracteres, y es determinista: el mismo fuente produce siempre el mismo nombre, aquí y en el entorno ASM.

9. Del `.bin` a KolibriOS: MOVER BINARIO y EJECUTAR

Compilar produce el ejecutable en tu disco. Probarlo de verdad pasa por estos dos botones, que son la otra mitad del Studio.

MOVER BINARIO

Instala el `.bin` del programa abierto dentro de la imagen persistente, en seis pasos que el registro de notificaciones va contando:

1. **Verificando imagen…** — `mdir` comprueba que la partición FAT32 de la imagen responde (desplazamiento 1048576).
2. **Copiando BIN a KolibriOS…** — `mcopy` incorpora el ejecutable a la raíz de la imagen con su nombre KolibriOS (§8), sin extensión.
3. **Preparando AUTORUN.DAT…** — se extrae el `AUTORUN.DAT` de `/KOLIBRIOS/SETTINGS/`.
4. **Configurando AUTORUN…** — se añade la línea `/SYS/<NOMBRE> "" 100` bajo el marcador `### Mini Entorno ASM ###`, solo si no estaba ya: mover dos veces el mismo programa **no duplica** su registro.
5. **Escribiendo AUTORUN.DAT en la imagen…** — se devuelve el archivo actualizado.
6. **Verificando imagen…** — se lista la raíz y la cola de `AUTORUN.DAT`. Si todo fue bien: `KOLIBRIOS LISTO`.

Gracias al paso 4, el programa **arranca solo** la próxima vez que KolibriOS inicie: es así como las pruebas de campo se hacen sin tocar nada dentro del sistema invitado.

Los temporales del proceso se limpian siempre, también cuando un paso falla — igual que el script Bash del que esta lógica procede.

EJECUTAR

Arranca la imagen en QEMU con el mismo comando del Mini Entorno ASM:

```
qemu-system-i386 -enable-kvm -cpu host -m 256 \
  -drive file=<imagen>,format=raw,if=ide -boot c -display gtk
```

QEMU queda en su propia sesión: cerrar el Studio no lo mata, y cerrar la ventana de QEMU no toca el Studio. El ciclo de campo completo es, entonces:

```
COMPILAR → MOVER BINARIO → EJECUTAR → KolibriOS arranca
           → AUTORUN lanza tu programa → observas su salida
```

Alcance del mover: *MOVER BINARIO mueve el ejecutable del programa abierto. Los archivos de datos que un programa lee (como el `FUENTE.KPP` del lexer) viajan a la imagen una vez, a mano con `mcopy`, y allí se quedan.*

10. Las guardas: cazando errores humanos

El Studio no supone que quien lo usa no se equivoca — ni siquiera si es su autor. Tres guardas vigilan el ciclo de KolibriOS, y cada una nació de un error real:

Un solo escritor sobre la imagen

La imagen es un disco FAT, y FAT no admite dos escritores. Si QEMU está abierto, KolibriOS guarda la FAT en caché con la foto vieja; si `mtools` escribe desde fuera en ese momento, las dos versiones se pisan y el sistema de archivos se rasga — pasó el 2026-11-11: un `AUTORUN.DAT` quedó con su entrada de directorio actualizada y su cadena de clusters sin aterrizar, ilegible.

Por eso, **con un QEMU abierto sobre la imagen**:

- **MOVER BINARIO se niega** a tocarla, y
- **EJECUTAR se niega** a abrir un segundo QEMU sobre ella.

El registro dice qué pasa y qué hacer: apagar KolibriOS desde su propio menú — QEMU se cierra solo y sus escrituras aterrizan completas — y volver a intentarlo. La disciplina queda así codificada en la herramienta: **entre corrida y corrida, KolibriOS cerrado; un solo escritor a la vez.**

EJECUTAR comprueba que el binario esté dentro

EJECUTAR solo arranca KolibriOS; no copia nada. Si el programa abierto aún no pasó por MOVER BINARIO, el botón lo comprueba (`mdir` sobre su nombre KolibriOS) y se niega a arrancar un KolibriOS donde ese programa no existe, recordando el orden del ciclo: **COMPILAR, MOVER BINARIO y EJECUTAR, en ese orden**. (Sin archivo abierto, la comprobación se salta: EJECUTAR también sirve para abrir KolibriOS a secas.)

Cada error trae su causa: la línea [DETALLE]

Cuando un paso del mover falla, el registro muestra el `[ERROR]` y debajo, en una línea `[DETALLE]`, la salida de la propia herramienta (`mdir`, `mcopy`): sus palabras exactas, no una interpretación. Un error sin causa no se puede diagnosticar; con la causa en el primer vistazo, la mayoría de los problemas de §11 se resuelven sin salir del Studio.

11. Solución de problemas

Síntoma	Causa probable	Solución
[ERROR] No existe K++: .../bin/kpp	El compilador no está construido	Repite el Paso 1 del §3
El diálogo ABRIR no muestra tus ejemplos	Estás en otra carpeta	Navega a projects/ de tu clon
DISASM ERROR tras compilar bien	Falta objdump o dd	Instala binutils y coreutils
La ventana no abre: no module named 'tkinter'	Falta el binding gráfico de Python	sudo apt install python3-tk
Error del compilador en K++ OUTPUT	Error en tu programa .kpp	Lee el mensaje (suele ser un ; olvidado o variable no declarada) y corrige en el editor
BIN INVÁLIDO / SALIDA INCOMPLETA	El pipeline no generó todos los artefactos	Revisa K++ OUTPUT buscando el primer error; prueba con un ejemplo conocido bueno (L01)
[ERROR] Falta el comando: mcopy / mdir / mtype	No está mtools	sudo apt install mtools
[ERROR] Falta el comando: qemu-system-i386	No está QEMU	sudo apt install qemu-system-x86
[ERROR] No existe la imagen persistente	La imagen no está en la ruta esperada	Colócala en la ruta por defecto o ajusta KPP_KOLIBRIOS_IMAGE (§3)
[ERROR] QEMU está abierto con esta imagen	La guarda de un solo escritor actuando (§10)	Apaga KolibriOS desde su menú y vuelve a intentarlo
[ERROR] ... todavía no está dentro de la imagen	EJECUTAR sin MOVER previo (§10)	Pulsa MOVER BINARIO y después EJECUTAR
[DETALLE] Fat problem while decoding... al extraer AUTORUN.DAT	La cadena de clusters de AUTORUN.DAT está rasgada (una escritura de KolibriOS no aterrizó; §10)	Cirugía de un archivo: extrae el AUTORUN.DAT de fábrica desde el .7z oficial, borra el roto de tu imagen con mdel y copia el sano con mcopy, siempre con QEMU cerrado. Luego MOVER vuelve a registrar tus programas
Tu programa no aparece dentro de KolibriOS	Buscas su nombre de fuente	Búscalo por su nombre KolibriOS (§8): la línea [KOLIBRIOS] de la compilación lo dice

12. Limitaciones actuales

- El botón **DEBUG** es un marcador de posición: el depurador llegará después del bootstrap. Por ahora, la verificación es `SOURCE → KPP → RAW/BIN → KolibriOS`.
- El Studio está pensado para Linux (o WSL): las rutas y herramientas (`dd`, `objdump`, `mtools`, QEMU) asumen ese entorno, y la guarda de un solo escritor inspecciona `/proc`.
- MOVER BINARIO instala el ejecutable del programa abierto; los archivos de datos se copian a la imagen por separado (§9).
- La ventana de QEMU es la del anfitrión (GTK): el Studio la lanza y la olvida; apagar KolibriOS es gesto del usuario, desde el menú del propio KolibriOS.
- El lenguaje que puedes testear es el documentado en el *Manual de Programación del Lenguaje K++*: Stage 0 completo y Stage 1 en construcción, cada pieza con su prueba de campo.

Guía elaborada a partir del código fuente real de `kpp_bootstrap_studio.py` (repositorio `github.com/saykol31/KPP`), en su estado tras el commit `368f481`. Complementa al Manual de Programación del Lenguaje K++.